NATIONAL UNIVERSITY OF SINGAPORE

CDE2605 Undergraduate Research Opportunities Programme
Academic Year 2025/2026 Semester 2

Project:
Intelligent Virtual Eye Screening Algorithm (iVESA) Using AI and VR

Final Project Report

NAME:
Kummarikunta Talitha (A0286653R)

Table of Contents

1. Abstract....3
2. Preface and Acknowledgements....4
3. Live Testing of Training Models....5
3.1. Additional Features....5
3.1.1. Display Features....5
3.1.2. Interface Workflow Features....9
3.1.2.1. Model Comaprison Feature....9
3.1.2.2. Add New Model Feature....10
3.1.2.3. Real-time Performance Evaluation Feature....11
3.1.2.3.1. Displayed under History Tab....11
3.1.2.3.2. Displayed under Models Tab....12
4. Analysis and Discussion....14
4.1. Limitations to be Addressed for Future Work....15
4.1.1. Processing Speed and System Architecture....15
4.1.2. GUI Improvements....17
4.1.3. Performance Metrics and Evaluation Depth....17
5. Conclusion....18
6. Appendix....18

1. Abstract

This research addresses a critical gap in the deployment of voice-enabled diagnostic systems by advancing the voice recognition platform within the Intelligent Virtual Eye Screening Algorithm (iVESA), a collaborative industrial initiative involving the Singapore Eye Research Institute, Singapore National Eye Centre, IOInspire PTE LTD, and the National University of Singapore (NUS). Conducted under the Undergraduate Research Opportunities Programme (UROP) 2026, the project focuses on strengthening the usability, scalability, and integration of voice-based functionalities within the iVESA ecosystem.

This paper details the development of a robust and extensible platform for the real-time testing, evaluation, and comparison of multiple voice recognition model iterations. The platform establishes a structured testing pipeline that supports live interaction, systematic data collection, and real-time performance feedback, enabling consistent and repeatable evaluation across evolving prototypes.

A central application of the platform is the evaluation of voice recognition systems for Singaporean children pronouncing letters of the English alphabet within automated eye screening workflows. By standardising testing procedures and improving user interaction with voice-enabled systems, this research enhances the reliability, accessibility, and deployment readiness of iVESA. Ultimately, the platform serves as a foundational component in accelerating the integration of voice-assisted technologies into real-world clinical practice.

2. Preface and Acknowledgements

I would like to express my sincere gratitude to Dr. Tang Kok Zuea and our industry collaborator, Mr. Harry Lim, for the opportunity to be involved in this research project at the National University of Singapore (NUS). Their guidance and support were invaluable throughout the course of this work.

I would also like to extend my appreciation to my teammate, Theertha Aravind, for her dedicated efforts in developing reliable voice recognition training models. Her commitment to iteratively training and refining each prototype played a crucial role in supporting the overall progress of the project.

3. Live Testing Platform for Training Models

This section introduces the key features and usage of the developed voice recognition testing platform, referred to as the "voice-to-text" application. The platform is an enhanced version of a conventional voice recorder, redesigned to support user-friendly, real-time testing of multiple iterations of voice recognition models. The graphical user interface (GUI) has been intentionally preserved from the original application, with additional functionalities integrated in a consistent manner to maintain an intuitive and accessible user experience.

The system facilitates efficient live recording and evaluation workflows, enabling users to seamlessly interact with and assess different model versions. Processing speed has been optimised to the extent currently achievable within the platform. However, further improvements—both at the platform level and within the underlying model architecture—remain possible and will be discussed in subsequent sections. Overall, the platform provides a streamlined and practical environment for iterative model testing and development.

3.1. Additional Features

3.1.1. Display Features

The setup instructions for the voice-to-text application are provided in the GitHub README. Users can follow these steps to install and launch the application. Upon initial launch, the interface remains consistent with the original voice recorder application (Figure 1), ensuring familiarity and ease of use.

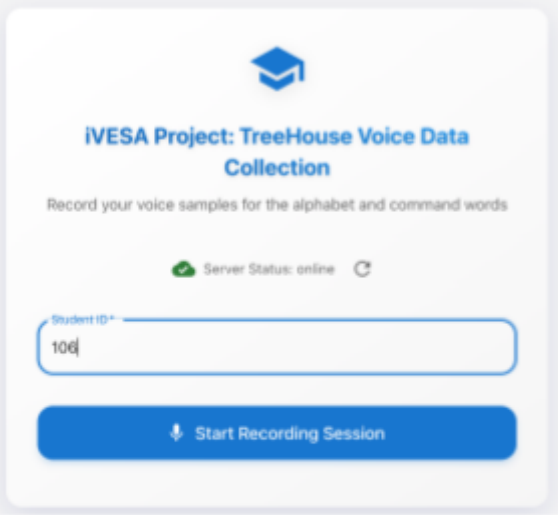

Figure 1. Opening Interface of app

To begin a session, the user inputs a Student ID and selects *"Start Recording Session."* This transitions the interface to the main testing environment (Figure 2), which includes the **Model Testing Panel**. This panel consists of:

1. a **Models Tab**, which displays all active models available for testing,
2. a **History Tab**, which records the evaluation results of each processed input, and
3. a **Compare Tab**, which enables direct comparison between selected models.

A default set of models is provided to allow users to quickly understand the interface and workflow.

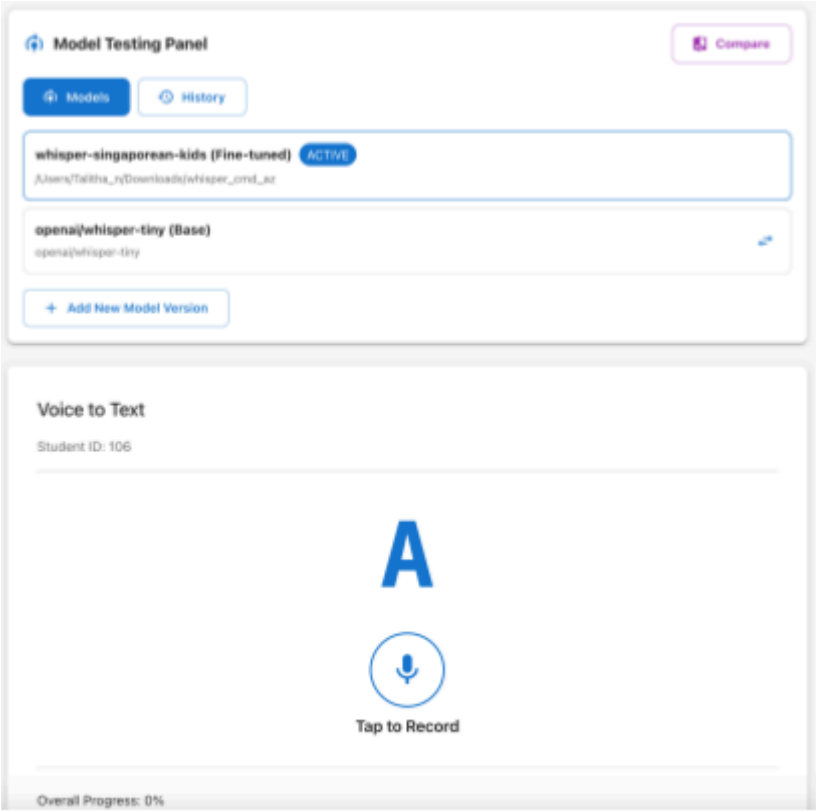

Figure 2. Recording Session Interface with updated features

To perform a recording, the user selects the *"Tap to Record"* icon, speaks the displayed letter, and presses the same icon again to stop recording (Figure 3).

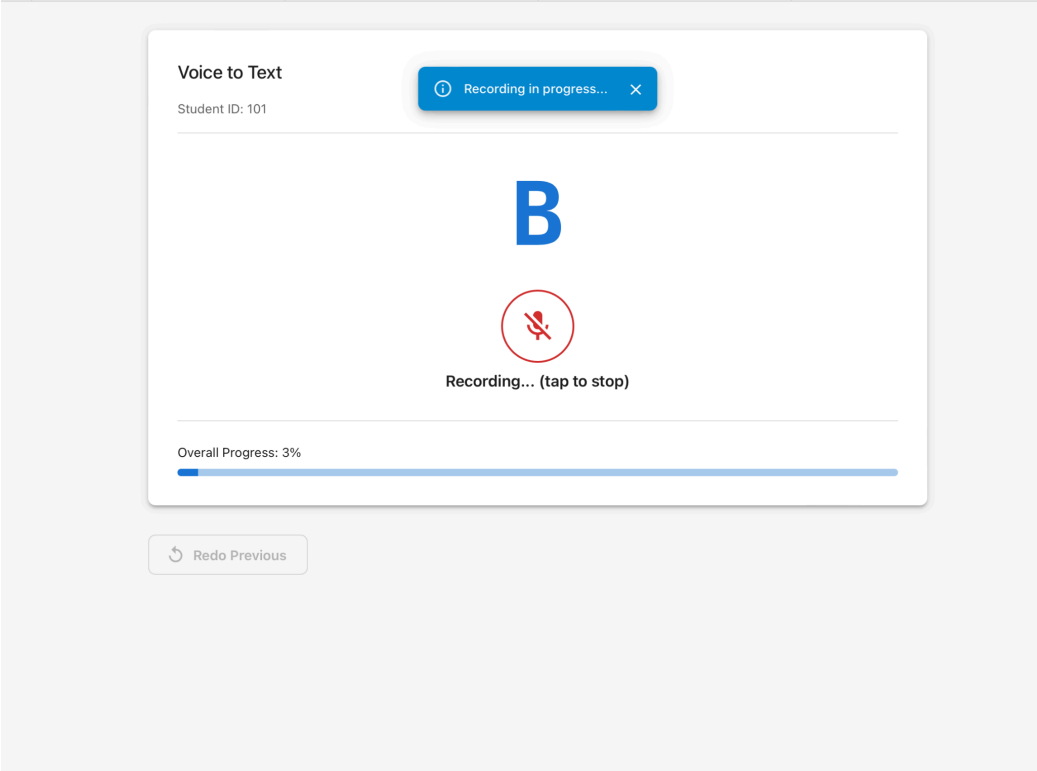

Figure 3. Display during Recording

The recorded input is then forwarded to the selected model(s) for processing (Figure 4). At present, this processing stage introduces noticeable latency due to model inference time, which is discussed further in the Limitations section. Additionally, the GUI label *"Analysing with Whisper model"* remains unchanged across models and can be disregarded during testing; this is identified as a potential improvement for future iterations.

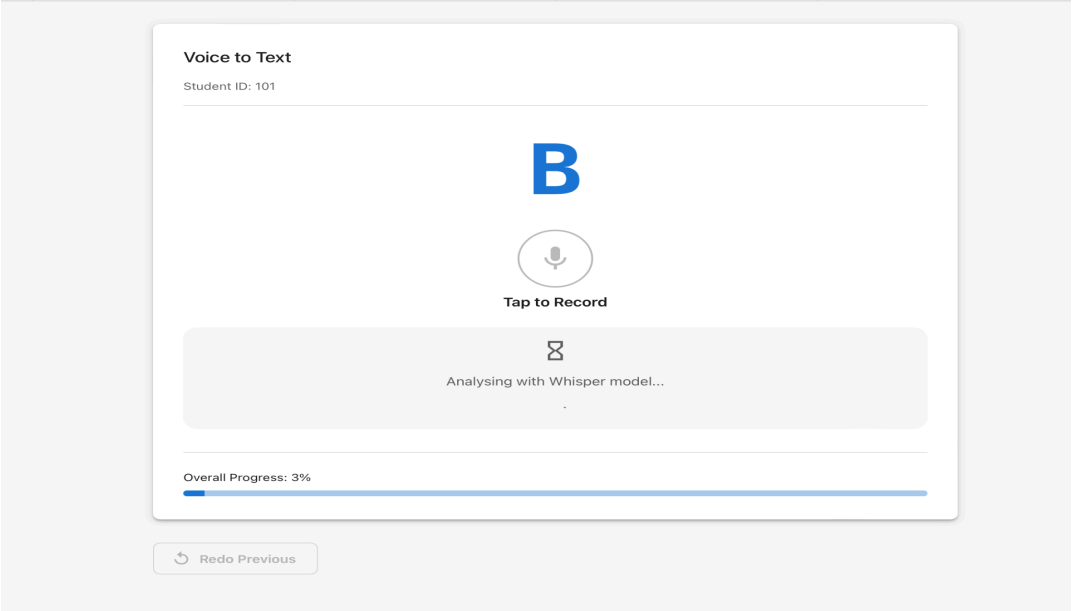

Figure 4. Display during Waiting of Model Outcome

The transcription result produced by the model is subsequently displayed (Figure 5).

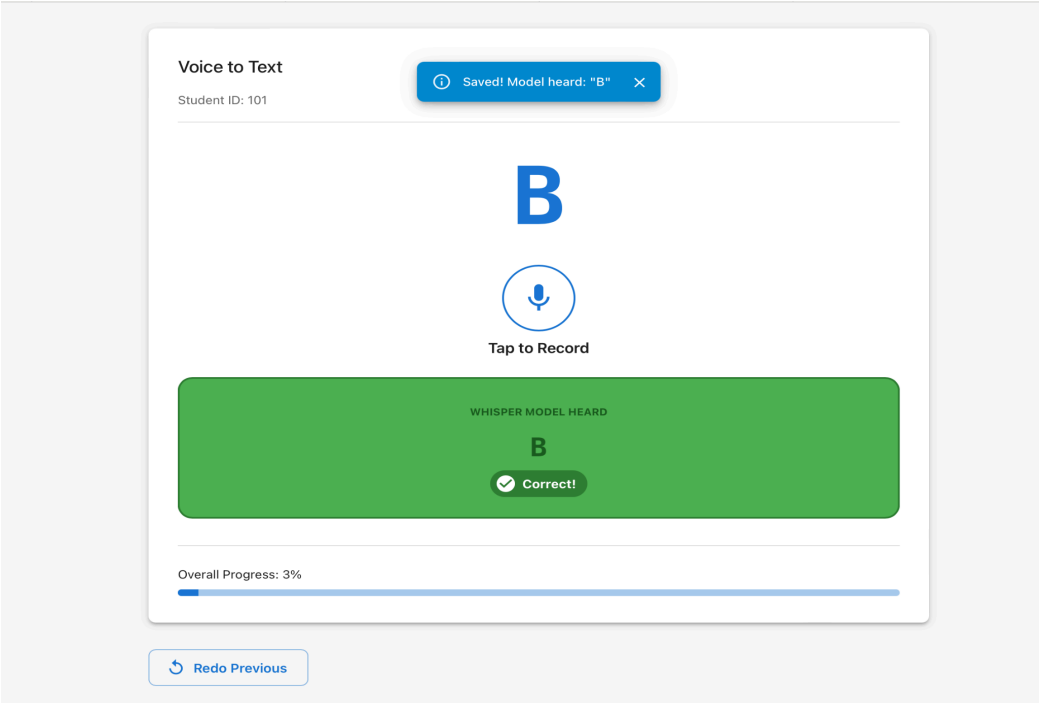

Figure 5. Display of Model Result

3.1.2. Interface Workflow Features

3.1.2.1. Model Comparison Feature

The model comparison feature enables real-time performance evaluation between two selected models. This is accessed via the **Compare Tab** in the Model Testing Panel (Figure 6).

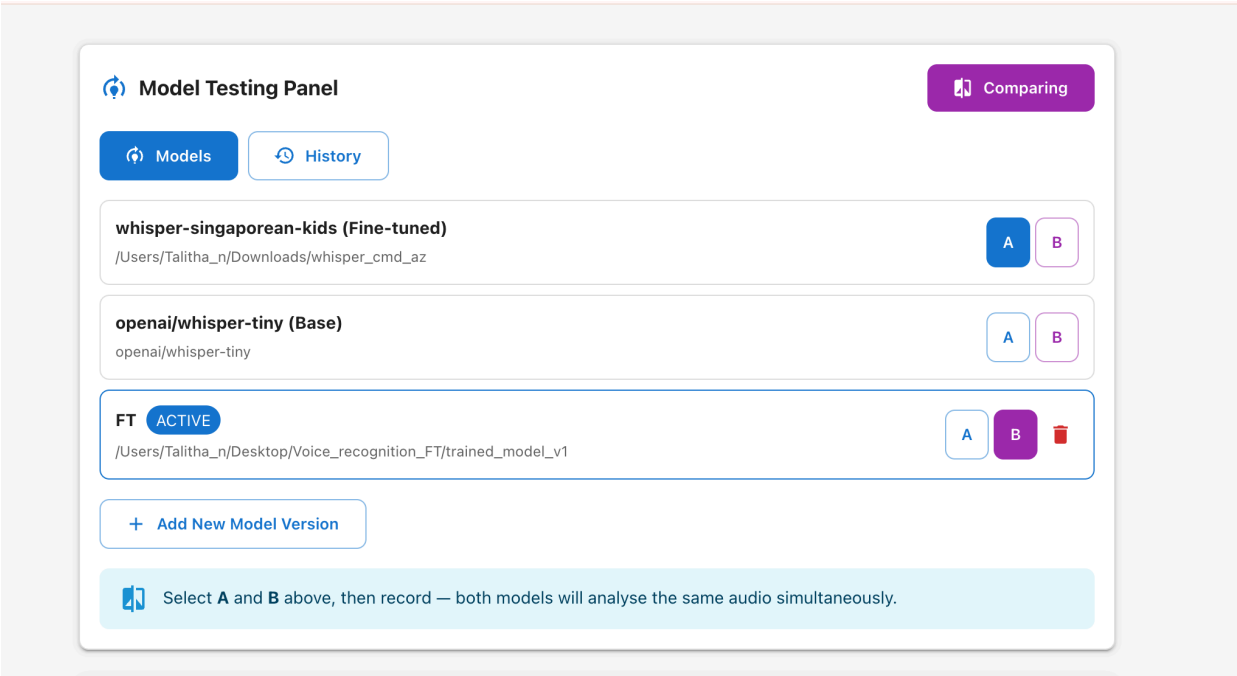

Figure 6. Comparison Tab Display

Each model can be assigned a label “A” or “B,” indicating its role in the comparison. If both labels are assigned to the same model, only a single model is effectively active. The results of the comparison are displayed as shown in Figure 7, allowing users to directly observe differences in model outputs and performance.

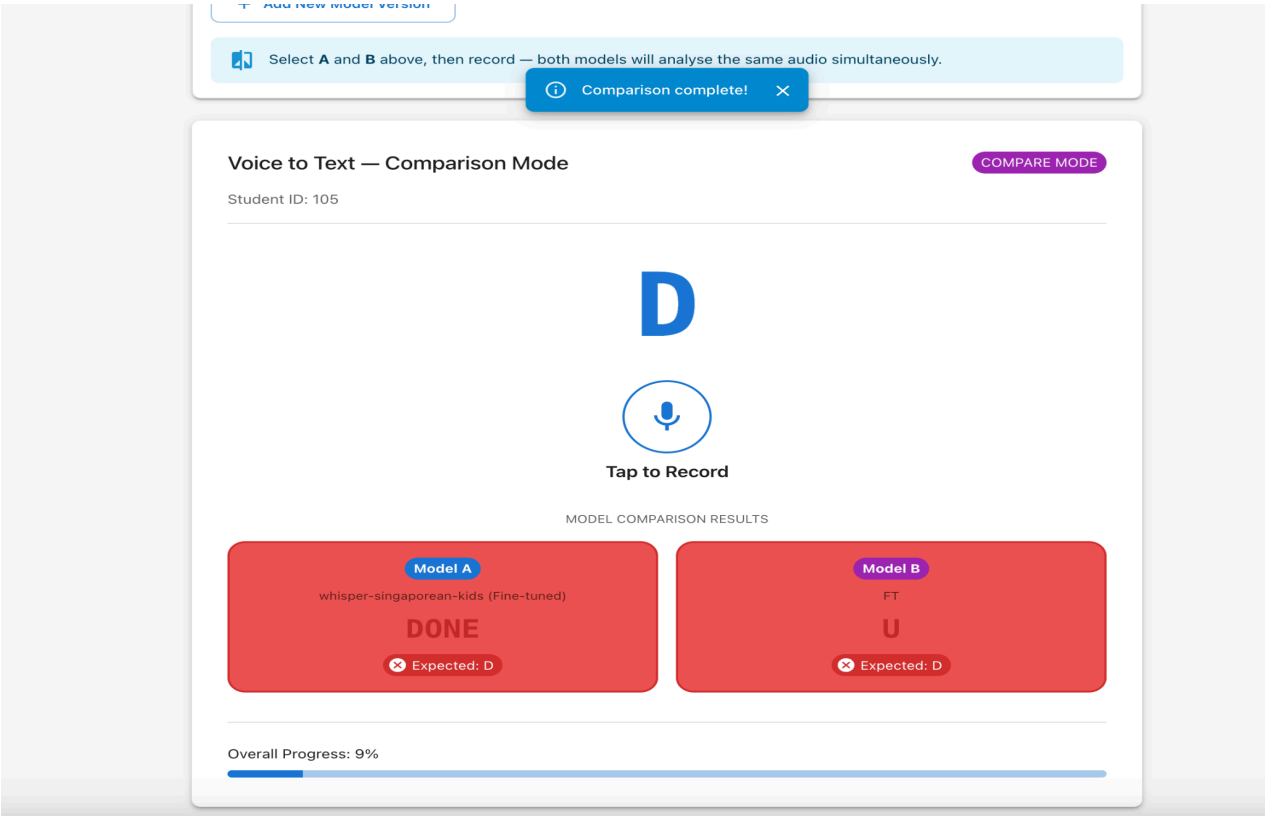

Figure 7. Result displayed in Compare Mode

3.1.2.2. Add New Model Feature

The platform supports the integration of new models through the *"Add New Model Version"* function (Figure 8). Users can specify:

1. a **Model Name** (user-defined), and
2. a **Model Path**, which must correspond to a valid trained model.

The model can be loaded either from a local directory or via a cloud-based source such as a Hugging Face model ID.

Users can switch between models using the double-arrow icon located on the right side of the interface. A delete (trash) icon is also provided to remove user-added models, while default models remain protected.

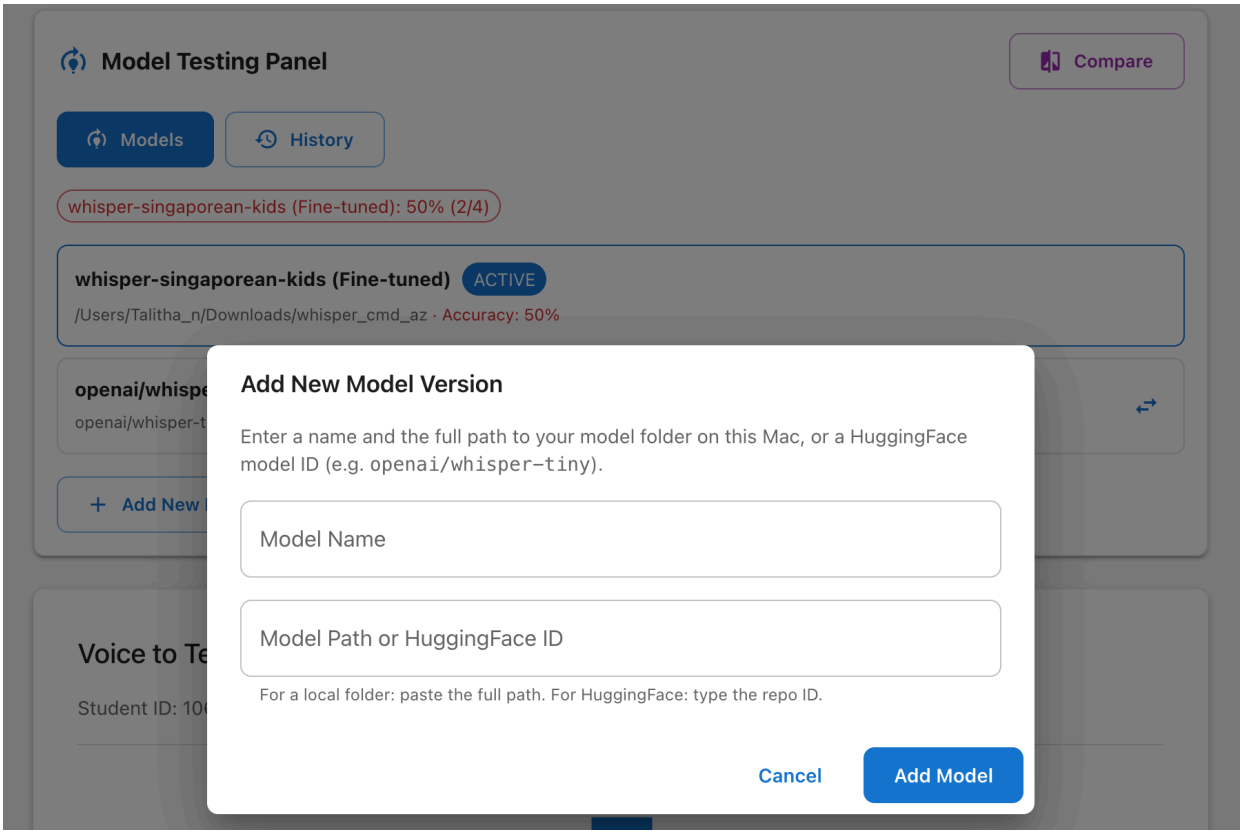

Figure 8. Add New Model Display

3.1.2.3. Real-time Performance Evaluation Feature

The platform incorporates a real-time evaluation mechanism that continuously updates model performance metrics as recordings are processed.

3.1.2.3.1. Displayed under History Tab

Displays a detailed log of all processed recordings, including the outputs generated by each model (Figure 9). This enables users to track performance across multiple test instances and analyse consistency over time.

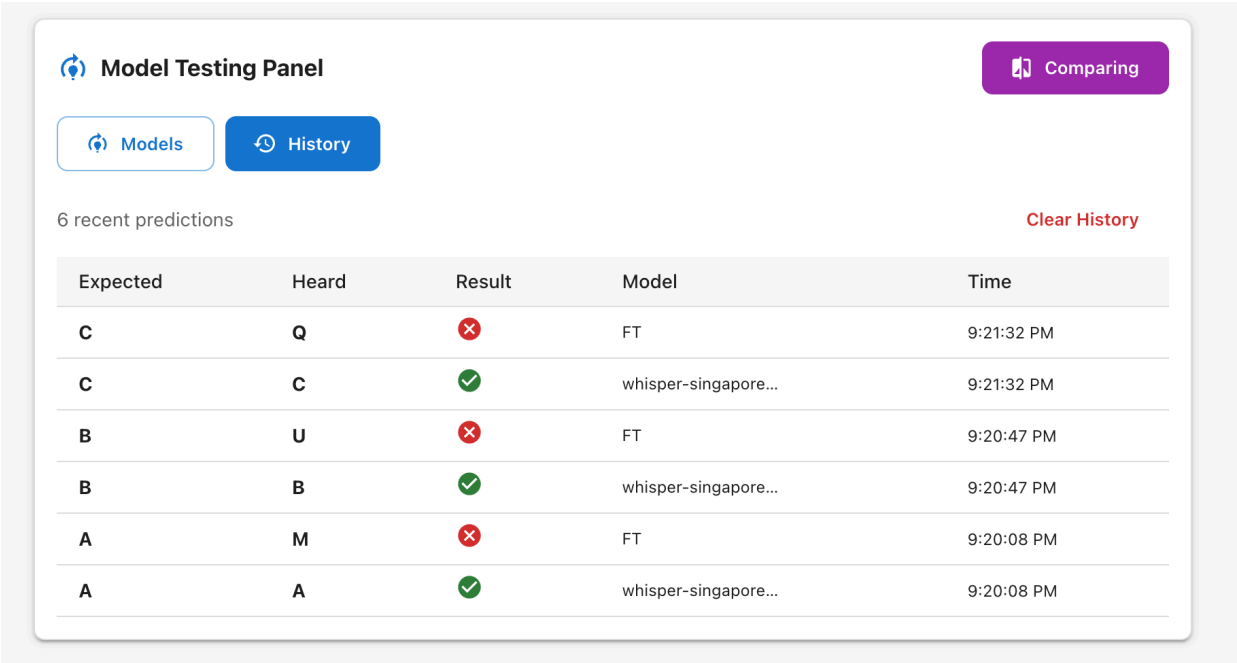

Figure 9. History Tab Display

3.1.2.3.2. Displayed under Models Tab

Provides a summary of each model's performance, including an updated accuracy metric displayed both at the top of the panel and alongside each model entry (Figure 10). This allows users to quickly assess overall model effectiveness during testing.

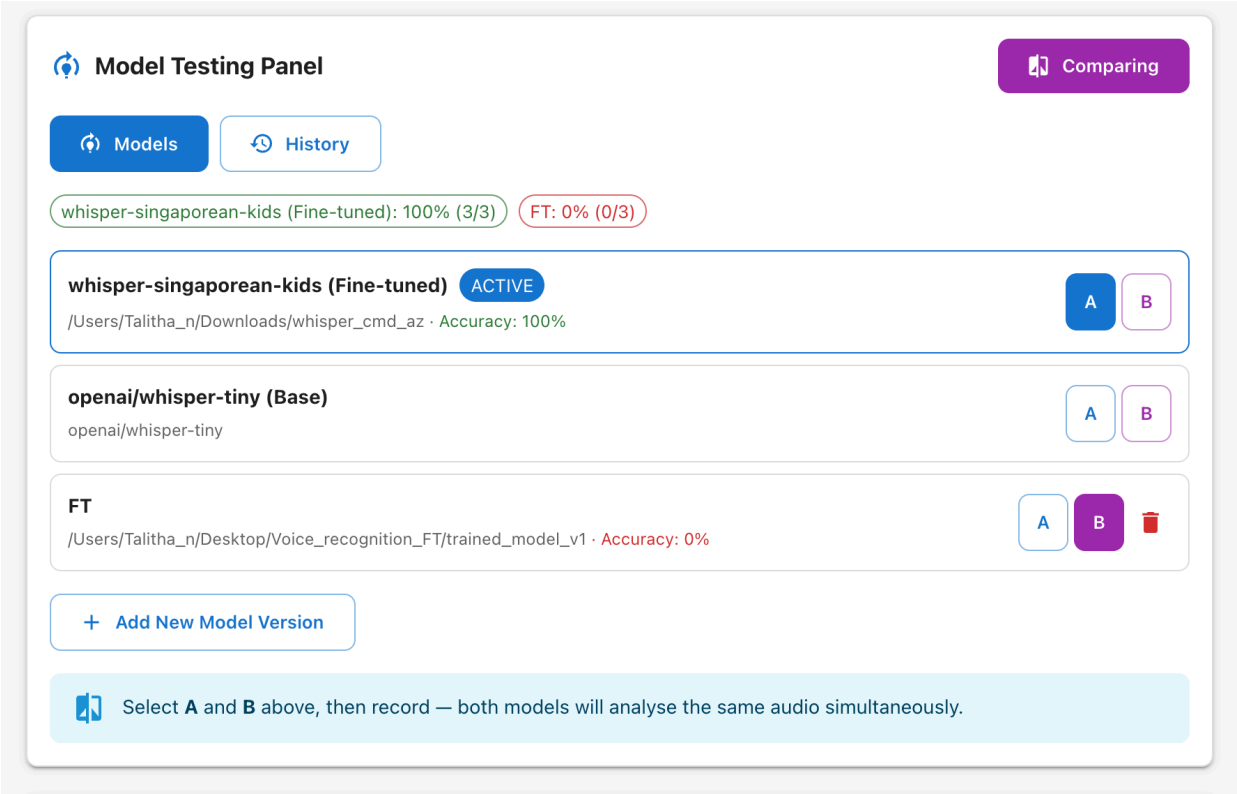

Figure 10. Model Evaluation

Overall, these features collectively establish a structured and interactive testing environment, enabling efficient evaluation, comparison, and iteration of voice recognition models within the iVESA workflow.

4. Analysis and Discussion

The developed voice-to-text platform demonstrates a significant improvement in enabling structured, real-time evaluation of multiple voice recognition models within a unified interface. By integrating model management, live recording, and performance tracking into a single workflow, the platform reduces the complexity typically associated with iterative model testing. This is particularly valuable in experimental settings where rapid comparison across model versions is required.

One of the key strengths of the system lies in its modular and extensible design. The ability to dynamically add, switch, and compare models allows for flexible experimentation without requiring modifications to the core application. This architecture not only supports current testing needs but also provides a strong foundation for future expansion, enabling additional functionalities such as advanced analytics, automated evaluation pipelines, and integration with external datasets or services. The inclusion of real-time performance tracking further enhances this capability by providing immediate feedback on model behaviour, allowing users to identify trends and inconsistencies across recordings.

The model comparison feature is especially impactful, as it transforms the platform from a simple testing interface into a decision-support tool. By allowing side-by-side evaluation, it facilitates more informed selection of models for deployment. Additionally, the structured logging of results in the History tab ensures reproducibility and traceability, which are critical in both research and clinical validation contexts. Importantly, this systematic data collection also enables downstream analysis, making the platform highly adaptable for future research questions and performance investigations.

However, while the platform is functionally robust, its performance and usability are influenced by several underlying constraints. These limitations highlight important areas for future improvement, particularly in terms of system efficiency, interface clarity, and evaluation depth.

4.1. Limitations to be Addressed for Future Work

4.1.1. Processing Speed and System Architechture

The most significant limitation of the current implementation is the processing latency during prediction. The root cause lies in the system architecture, where each prediction request spawns a new Python process. This results in repeated overhead for every recording, including:

1. Initialisation of the Python interpreter
2. Reloading of heavy libraries (e.g., TensorFlow, PyTorch, Librosa)
3. Re-loading of the trained model from disk

As a result, each prediction incurs an additional delay of approximately 3–5 seconds for Whisper-based models and 1–2 seconds for Keras-based models, independent of actual inference time.

To address this, a transition to a **persistent model server architecture** is proposed. Instead of launching a new process for each prediction, a local server (e.g., Flask-based) can be used to load the model once and handle all subsequent prediction requests via HTTP (Figures 11 and 12).

```python
#!/usr/bin/env python3
"""
model_server.py - Persistent prediction server.
Loads the model ONCE and serves predictions over HTTP.
Usage: python model_server.py <model_dir> [--port 5001]
"""

from flask import Flask, request, jsonify
import argparse
import sys
import os

# Reuse all the logic from predict.py
sys.path.insert(0, os.path.dirname(__file__))
from predict import detect_model_type, MODEL_BACKENDS, map_to_allowed, normalize_label

app = Flask(__name__)

# Global - loaded once at startup
MODEL_TYPE   = None
INFER_FN     = None
MODEL_DIR    = None

@app.route('/predict', methods=['POST'])
def predict_endpoint():
    data       = request.json
    audio_path = data.get('audio_path')

    if not audio_path or not os.path.exists(audio_path):
        return jsonify({"error": "audio file not found"}), 400

    try:
        raw_text = INFER_FN(audio_path, MODEL_DIR)
        pred     = map_to_allowed(raw_text)
        return jsonify({"label": pred})
    except Exception as e:
        return jsonify({"error": str(e)}), 500
```

Figure 11. Model Server Code

```python
@app.route('/health', methods=['GET'])
def health():
    return jsonify({"status": "ok", "model_type": MODEL_TYPE, "model_dir": MODEL_DIR})

@app.route('/switch_model', methods=['POST'])
def switch_model():
    """Hot-swap the model without restarting the server."""
    global MODEL_TYPE, INFER_FN, MODEL_DIR
    data      = request.json
    new_dir   = data.get('model_dir')
    if not new_dir or not os.path.exists(new_dir):
        return jsonify({"error": "model_dir not found"}), 400
    MODEL_TYPE = detect_model_type(new_dir)
    INFER_FN   = MODEL_BACKENDS.get(MODEL_TYPE)
    MODEL_DIR  = new_dir
    return jsonify({"status": "switched", "model_type": MODEL_TYPE})

if __name__ == '__main__':
    parser = argparse.ArgumentParser()
    parser.add_argument('model_dir')
    parser.add_argument('--port', type=int, default=5001)
    args = parser.parse_args()

    MODEL_DIR  = args.model_dir
    MODEL_TYPE = detect_model_type(MODEL_DIR)
    INFER_FN   = MODEL_BACKENDS.get(MODEL_TYPE)

    if not INFER_FN:
        print(f"[ERROR] Unknown model type in {MODEL_DIR}")
        sys.exit(1)

    # Warm up — load model into memory now
    print(f"[SERVER] Loading {MODEL_TYPE} model from {MODEL_DIR}...")
    print(f"[SERVER] Ready on port {args.port}")
    app.run(port=args.port, debug=False)
```

Figure 12. Model Server Code

This architectural shift has already demonstrated measurable improvements, reducing total processing time from approximately 45 seconds to 20 seconds per session. However, the first prediction still incurs a longer delay due to initial model loading, after which subsequent predictions benefit from reduced latency.

Further optimisations include:

1. Preloading libraries at the start of execution to reduce initial delay,
2. Converting Keras models to TensorFlow Lite (TFLite) format, which can achieve 3–5× faster inference with reduced model size while maintaining comparable accuracy.

These enhancements collectively represent a critical step toward achieving real-time responsiveness required for practical deployment.

4.1.2. GUI Improvements

While the platform maintains a consistent and intuitive interface, several aspects of the GUI can be further refined to improve usability. For instance, the current display message *"Analysing with Whisper model"* remains static regardless of the model being used. This may lead to confusion during testing, particularly when multiple models are evaluated simultaneously.

Future iterations should incorporate dynamic UI elements that accurately reflect the active model and processing state. Additional enhancements could include clearer visual feedback during recording and processing, improved status indicators, and more intuitive navigation within the Model Testing Panel. These refinements would strengthen user confidence and reduce potential misinterpretation of results.

4.1.3. Performance Metrics and Evaluation Depth

The current evaluation framework primarily focuses on accuracy as the key performance metric. While this provides a useful high-level indication of model performance, it does not capture the full range of behaviours necessary for comprehensive evaluation.

Future work could expand the evaluation framework to include additional metrics such as:

1. Precision and recall
2. Confusion matrices for letter classification
3. Latency per prediction
4. Robustness across varying audio conditions (e.g., noise levels, accents)

Incorporating these metrics would provide a more holistic understanding of model performance and enable more informed comparisons between models.

5. Conclusion

This project presents the development of a comprehensive voice recognition testing platform designed to support the iterative evaluation of multiple model versions within the iVESA framework. By integrating real-time recording, model management, comparison tools, and performance tracking into a unified interface, the platform significantly enhances the efficiency and structure of the model testing process.

The system addresses a key bottleneck in experimental workflows by enabling consistent and repeatable evaluation across evolving prototypes. While limitations related to processing speed, interface clarity, and evaluation depth remain, the proposed improvements—particularly the transition to a persistent model server—demonstrate a clear pathway toward achieving real-time, scalable performance.

Overall, this work establishes a strong foundation for the integration of voice-enabled technologies into automated eye screening systems. By improving both the usability and robustness of the testing pipeline, the platform contributes to the broader goal of developing reliable, accessible, and clinically deployable diagnostic solutions.

6. Appendix

[1] Kummarikunta, T. (2026) *"Voice-to-text" Application Software*. GitHub. Retrieved from https://github.com/Tali-22/TreeHouseDataCollection_modified.git

[2] Park, S. (2025). Data Collection Software for TreeHouse Trials. GitHub. Retrieved from https://github.com/sampark0523/TreeHouseDataCollection.

[3] Park, S. (2025). Fine-tuning Whisper for Singaporean Children's Speech. Google Colab Notebook. Retrieved from https://colab.research.google.com/drive/1eBBA_whyq5yguZJBt7d8LnvLXsg3MPZL.

[4] Park, S. (2025). whisper-singaporean-kids. Hugging Face. Retrieved from https://huggingface.co/sampark0523/whisper-singaporean-kids.

[5] Park, S. (2025). Fourier Transform Training Model.